

Hadoop & HDFS Complete Guide (Windows)

This document merges both source documents into one reference without intentionally removing useful content.

The standard installation flow comes first, followed by the non-administrator alternatives and then all programming examples.

Hadoop & HDFS: Installation, Configuration & Exploration on Windows

1. Installing Hadoop on Windows

Prerequisites

- **Java JDK 8 or 11** (required by Hadoop)
- **Windows 10/11** (64-bit)
- **Hadoop binary** (version 3.3.x recommended)
- **Hadoop Windows utilities** (winutils.exe and hadoop.dll)

Step-by-Step Installation

Step 1: Install Java JDK

```
# Verify Java installation after installing JDK
java -version
```

Set the environment variable:

```
JAVA_HOME = C:\Program Files\Java\jdk1.8.0_xxx
```

Note: The path must NOT contain spaces when referenced by Hadoop. Use the short path `C:\Progra~1\Java\jdk1.8.0_xxx` if needed.

Step 2: Download Hadoop

Download **Hadoop 3.3.6** from the Apache archive. Extract it to a simple path:

```
C:\hadoop
```

Your directory structure should look like:

```
C:\hadoop\bin
C:\hadoop\etc\hadoop
C:\hadoop\sbin
C:\hadoop\share
...
```

Step 3: Download Windows Utilities

Download `winutils.exe` and `hadoop.dll` matching your Hadoop version from the GitHub repository (search for "winutils hadoop 3.3"). Place them in:

```
C:\hadoop\bin\winutils.exe
C:\hadoop\bin\hadoop.dll
```

Also copy `hadoop.dll` to:

```
C:\Windows\System32\hadoop.dll
```

Step 4: Set Environment Variables

Variable	Value
<code>HADOOP_HOME</code>	<code>C:\hadoop</code>
<code>JAVA_HOME</code>	<code>C:\Progra~1\Java\jdk1.8.0_xxx</code>
Add to <code>PATH</code>	<code>%HADOOP_HOME%\bin;%HADOOP_HOME%\sbin</code>

Verify:

```
hadoop version
```

2. Configuring Hadoop and HDFS

All configuration files are in `C:\hadoop\etc\hadoop\`.

A. core-site.xml

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
  </property>
  <property>
    <name>hadoop.tmp.dir</name>
    <value>C:/hadoop/tmp</value>
  </property>
</configuration>
```

B. hdfs-site.xml

```
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
  <property>
    <name>dfs.namenode.name.dir</name>
    <value>file:///C:/hadoop/data/namenode</value>
  </property>
  <property>
    <name>dfs.datanode.data.dir</name>
    <value>file:///C:/hadoop/data/datanode</value>
  </property>
</configuration>
```

C. mapred-site.xml

```
<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
</configuration>
```

D. yarn-site.xml

```
<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
</configuration>
```

E. hadoop-env.cmd

Edit and set:

```
set JAVA_HOME=C:\Progra~1\Java\jdk1.8.0_xxx
```

Format NameNode & Start Services

```
# Format the NameNode (only once, first time)
hdfs namenode -format

# Start HDFS
start-dfs.cmd

# Start YARN (optional, for MapReduce)
start-yarn.cmd
```

Verify services by opening the web UI:

Service	URL
NameNode	http://localhost:9870
DataNode	http://localhost:9864
YARN	http://localhost:8088

3. HDFS Shell Commands (Demonstration)

Basic HDFS Commands Reference

Command	Description
<code>hdfs dfs -ls /</code>	List root directory
<code>hdfs dfs -mkdir /dirname</code>	Create directory
<code>hdfs dfs -put local remote</code>	Upload file
<code>hdfs dfs -get remote local</code>	Download file
<code>hdfs dfs -cat /path/file</code>	Read/display file
<code>hdfs dfs -rm /path/file</code>	Delete file
<code>hdfs dfs -rmdir /path/dir</code>	Remove empty directory
<code>hdfs dfs -cp src dest</code>	Copy file in HDFS
<code>hdfs dfs -mv src dest</code>	Move/rename file
<code>hdfs dfs -chmod 755 /path</code>	Change permissions
<code>hdfs dfs -du /path</code>	Check disk usage
<code>hdfs dfs -tail /path/file</code>	Show last 1KB
<code>hdfs dfs -touchz /path/file</code>	Create empty file
<code>hdfs dfs -count /path</code>	Count files/dirs/bytes

Full Demonstration

```
# ===== CREATE DIRECTORIES =====
hdfs dfs -mkdir /user
hdfs dfs -mkdir /user/mydata
hdfs dfs -mkdir -p /user/mydata/subfolder

# ===== UPLOAD FILES (PUT) =====
# Create a local test file first
echo "Hello Hadoop HDFS World" > C:\temp\sample.txt

# Upload to HDFS
hdfs dfs -put C:\temp\sample.txt /user/mydata/
# OR use copyFromLocal
hdfs dfs -copyFromLocal C:\temp\sample.txt /user/mydata/sample2.txt

# ===== LIST FILES =====
```

```
hdfs dfs -ls /user/mydata/
hdfs dfs -ls -R /user/          # Recursive listing

# ===== READ/DISPLAY FILES =====
hdfs dfs -cat /user/mydata/sample.txt
hdfs dfs -tail /user/mydata/sample.txt
hdfs dfs -head /user/mydata/sample.txt

# ===== DOWNLOAD FILES (GET) =====
hdfs dfs -get /user/mydata/sample.txt C:\temp\downloaded.txt
# OR use copyToLocal
hdfs dfs -copyToLocal /user/mydata/sample.txt C:\temp\downloaded2.txt

# ===== COPY & MOVE WITHIN HDFS =====
hdfs dfs -cp /user/mydata/sample.txt /user/mydata/subfolder/
hdfs dfs -mv /user/mydata/sample2.txt /user/mydata/subfolder/renamed.txt

# ===== FILE INFO & STATS =====
hdfs dfs -du /user/mydata/
hdfs dfs -du -h /user/mydata/      # Human-readable sizes
hdfs dfs -count /user/mydata/
hdfs dfs -stat "%b %n %r" /user/mydata/sample.txt

# ===== PERMISSIONS =====
hdfs dfs -chmod 755 /user/mydata/sample.txt
hdfs dfs -chown myuser:mygroup /user/mydata/sample.txt

# ===== MERGE FILES =====
hdfs dfs -getmerge /user/mydata/ C:\temp\merged_output.txt

# ===== DELETE FILES =====
hdfs dfs -rm /user/mydata/subfolder/renamed.txt
hdfs dfs -rm -r /user/mydata/subfolder/      # Recursive delete
hdfs dfs -rmdir /user/mydata/subfolder/      # Remove empty dir

# ===== CHECK HDFS HEALTH =====
hdfs dfsadmin -report
hdfs fsck /user/mydata/ -files -blocks
```

4. Working with HDFS in Python

Install Required Libraries

```
pip install hdfs
pip install pydoop          # Alternative (Linux-friendly)
pip install pyarrow         # For parquet on HDFS
```

Python Code: HDFS Operations using **hdfs** library

```
from hdfs import InsecureClient

# ===== CONNECT TO HDFS =====
# Connect via WebHDFS (NameNode REST API on port 9870)
client = InsecureClient('http://localhost:9870', user='hadoop')

# ===== CREATE DIRECTORY =====
client.makedirs('/user/python_data')
print("Directory created successfully")

# ===== UPLOAD FILE =====
# Upload a local file to HDFS
client.upload('/user/python_data/sample.txt', r'C:\temp\sample.txt')
print("File uploaded successfully")

# ===== LIST FILES =====
file_list = client.list('/user/python_data/')
print("Files in HDFS directory:", file_list)

# ===== READ FILE =====
with client.read('/user/python_data/sample.txt', encoding='utf-8') as reader:
    content = reader.read()
    print("File Content:", content)

# ===== DOWNLOAD FILE =====
client.download('/user/python_data/sample.txt', r'C:\temp\python_download')
print("File downloaded successfully")

# ===== WRITE DATA DIRECTLY =====
data = "This is written directly from Python to HDFS!\nLine 2 of data."
client.write('/user/python_data/direct_write.txt', data=data, encoding='utf-8')
print("Data written directly to HDFS")

# ===== APPEND TO FILE =====
client.write('/user/python_data/direct_write.txt',
            data="\nAppended line from Python",
```

```

        append=True, encoding='utf-8')
print("Data appended successfully")

# ===== GET FILE STATUS =====
status = client.status('/user/python_data/sample.txt')
print("File Status:", status)

# ===== RENAME/MOVE FILE =====
client.rename('/user/python_data/direct_write.txt',
              '/user/python_data/renamed_file.txt')
print("File renamed successfully")

# ===== DELETE FILE =====
client.delete('/user/python_data/renamed_file.txt')
print("File deleted successfully")

# ===== DELETE DIRECTORY (Recursive) =====
client.delete('/user/python_data', recursive=True)
print("Directory deleted successfully")

```

Python: Using Pandas with HDFS

```

import pandas as pd
from hdfs import InsecureClient

client = InsecureClient('http://localhost:9870', user='hadoop')

# ===== WRITE DataFrame TO HDFS =====
df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana'],
    'Age': [25, 30, 35, 28],
    'Department': ['HR', 'IT', 'Finance', 'IT']
})

# Write CSV to HDFS
with client.write('/user/python_data/employees.csv', encoding='utf-8') as \
    df.to_csv(writer, index=False)
print("DataFrame written to HDFS as CSV")

# ===== READ DataFrame FROM HDFS =====
with client.read('/user/python_data/employees.csv', encoding='utf-8') as \
    df_read = pd.read_csv(reader)

```



```
print("DataFrame read from HDFS:")
print(df_read)
```

5. Working with HDFS in R

Install Required Libraries

```
# Install packages
install.packages("rhdfs")          # Direct HDFS access (requires rJava)
install.packages("rjava")
install.packages("httr")          # For WebHDFS REST API
install.packages("jsonlite")     # JSON parsing

# Alternative: ravro for Hadoop data formats
install.packages("sparklyr")     # Spark interface (includes HDFS access)
```

Method 1: R with WebHDFS REST API (Recommended for Windows)

```
library(httr)
library(jsonlite)

# ===== CONFIGURATION =====
namenode_url <- "http://localhost:9870"
hdfs_user <- "hadoop"
webhdfs_base <- paste0(namenode_url, "/webhdfs/v1")

# ===== LIST DIRECTORY =====
list_hdfs <- function(path) {
  url <- paste0(webhdfs_base, path, "?op=LISTSTATUS&user.name=", hdfs_user)
  response <- GET(url)
  content <- fromJSON(content(response, "text", encoding = "UTF-8"))
  files <- content$FileStatuses$FileStatus
  return(files)
}

files <- list_hdfs("/user/")
print(files)

# ===== CREATE DIRECTORY =====
```

```

create_dir_hdfs <- function(path) {
  url <- paste0(webhdfs_base, path, "?op=MKDIRS&user.name=", hdfs_user)
  response <- PUT(url)
  result <- fromJSON(content(response, "text", encoding = "UTF-8"))
  return(result$boolean)
}

```

```

create_dir_hdfs("/user/r_data")
cat("Directory created successfully\n")

```

```

# ===== UPLOAD FILE =====

```

```

upload_to_hdfs <- function(local_path, hdfs_path) {
  # Step 1: Get redirect URL
  url <- paste0(webhdfs_base, hdfs_path,
                "?op=CREATE&overwrite=true&user.name=", hdfs_user)
  response <- PUT(url, config = config(followlocation = FALSE))

  # Step 2: Upload to DataNode
  redirect_url <- headers(response)$location
  file_data <- upload_file(local_path)
  response2 <- PUT(redirect_url, body = file_data)
  return(status_code(response2))
}

```

```

upload_to_hdfs("C:/temp/sample.txt", "/user/r_data/sample.txt")
cat("File uploaded successfully\n")

```

```

# ===== READ FILE =====

```

```

read_hdfs_file <- function(hdfs_path) {
  url <- paste0(webhdfs_base, hdfs_path,
                "?op=OPEN&user.name=", hdfs_user)
  response <- GET(url)
  content <- content(response, "text", encoding = "UTF-8")
  return(content)
}

```

```

file_content <- read_hdfs_file("/user/r_data/sample.txt")
cat("File Content:", file_content, "\n")

```

```

# ===== GET FILE STATUS =====

```

```

get_file_status <- function(hdfs_path) {
  url <- paste0(webhdfs_base, hdfs_path,
                "?op=GETFILESTATUS&user.name=", hdfs_user)
  response <- GET(url)
}

```

```

status <- fromJSON(content(response, "text", encoding = "UTF-8"))
return(status$FileStatus)
}

status <- get_file_status("/user/r_data/sample.txt")
print(status)

# ===== DOWNLOAD FILE =====
download_from_hdfs <- function(hdfs_path, local_path) {
  content <- read_hdfs_file(hdfs_path)
  writeLines(content, local_path)
  cat("Downloaded to:", local_path, "\n")
}

download_from_hdfs("/user/r_data/sample.txt", "C:/temp/r_downloaded.txt")

# ===== DELETE FILE =====
delete_hdfs <- function(hdfs_path, recursive = FALSE) {
  url <- paste0(webhdfs_base, hdfs_path,
    "?op=DELETE&recursive=", tolower(as.character(recursive))
    "&user.name=", hdfs_user)
  response <- DELETE(url)
  result <- fromJSON(content(response, "text", encoding = "UTF-8"))
  return(result$boolean)
}

delete_hdfs("/user/r_data/sample.txt")
cat("File deleted\n")

delete_hdfs("/user/r_data", recursive = TRUE)
cat("Directory deleted\n")

```

Method 2: R using `system()` for HDFS Shell Commands

```

# ===== WRAPPER FUNCTION =====
hdfs_cmd <- function(command) {
  full_cmd <- paste("hdfs dfs", command)
  result <- system(full_cmd, intern = TRUE)
  cat(paste(result, collapse = "\n"), "\n")
  return(result)
}

```

```
# ===== DEMONSTRATE ALL OPERATIONS =====

# List root directory
hdfs_cmd("-ls /")

# Create directory
hdfs_cmd("-mkdir -p /user/r_test")

# Upload file
hdfs_cmd("-put C:/temp/sample.txt /user/r_test/")

# List files
hdfs_cmd("-ls /user/r_test/")

# Read file content
hdfs_cmd("-cat /user/r_test/sample.txt")

# Get file info
hdfs_cmd("-du -h /user/r_test/")

# Copy file within HDFS
hdfs_cmd("-cp /user/r_test/sample.txt /user/r_test/copy.txt")

# Download file
hdfs_cmd("-get /user/r_test/sample.txt C:/temp/r_output.txt")

# Delete files
hdfs_cmd("-rm /user/r_test/copy.txt")
hdfs_cmd("-rm -r /user/r_test")
```

Method 3: R with DataFrames on HDFS

```
library(httr)
library(jsonlite)

namenode_url <- "http://localhost:9870"
hdfs_user <- "hadoop"
webhdfs_base <- paste0(namenode_url, "/webhdfs/v1")

# ===== WRITE DATAFRAME TO HDFS AS CSV =====
write_df_to_hdfs <- function(df, hdfs_path) {
  # Convert dataframe to CSV string
```

```

tmp_file <- tempfile(fileext = ".csv")
write.csv(df, tmp_file, row.names = FALSE)

# Upload to HDFS
upload_to_hdfs(tmp_file, hdfs_path)
unlink(tmp_file)
cat("DataFrame written to HDFS:", hdfs_path, "\n")
}

# ===== READ DATAFRAME FROM HDFS =====
read_df_from_hdfs <- function(hdfs_path) {
  content <- read_hdfs_file(hdfs_path)
  df <- read.csv(text = content)
  return(df)
}

# Example usage
employees <- data.frame(
  Name = c("Alice", "Bob", "Charlie", "Diana"),
  Age = c(25, 30, 35, 28),
  Department = c("HR", "IT", "Finance", "IT"),
  stringsAsFactors = FALSE
)

# Write DataFrame
write_df_to_hdfs(employees, "/user/r_data/employees.csv")

# Read DataFrame back
df_from_hdfs <- read_df_from_hdfs("/user/r_data/employees.csv")
print(df_from_hdfs)

```

Troubleshooting Tips

- **"JAVA_HOME not set"** – Ensure the path has no spaces; use `C:\Progra~1\...`
- **"Connection refused on port 9000"** – Ensure `start-dfs.cmd` ran successfully
- **"winutils.exe not found"** – Confirm `HADOOP_HOME\bin\winutils.exe` exists
- **Permission errors** – Run CMD as Administrator; check `hadoop.tmp.dir` folder exists
- **NameNode won't start** – Delete `C:\hadoop\data\namenode` and re-format with `hdfs namenode -format`

Would you like me to elaborate on any specific section, such as setting up a multi-node cluster, integrating with Spark, or handling specific file formats like Parquet or Avro?

Appendix A – Installing Hadoop Without Administrator Access

The following section complements the main guide. Use it only if you cannot modify system settings or install software under `C:\`.

1. Set User-Level Environment Variables (No Admin Needed)

Method A: Via Windows Settings (User Variables)

- 1. Press `Win + S` → search **"Environment Variables"**
- 2. Click **"Edit environment variables for your account"** (this does NOT require admin)
- 3. Under **"User variables for [your_username]"**, click **New**:

Variable	Value
<code>JAVA_HOME</code>	<code>C:\Users\YourName\java\jdk1.8.0_xxx</code>
<code>HADOOP_HOME</code>	<code>C:\Users\YourName\hadoop</code>

- 4. Edit the **user-level Path** variable and add:

```
%HADOOP_HOME%\bin
%HADOOP_HOME%\sbin
%JAVA_HOME%\bin
```

Key Point: User-level variables do NOT require admin access — only **System variables** do.

Method B: Set Variables via Command Line (Per Session)

If even user-level GUI is restricted, set variables **temporarily in each CMD session**:

```
@echo off
REM ===== Set variables for this session =====
set JAVA_HOME=C:\Users\YourName\java\jdk1.8.0_xxx
set HADOOP_HOME=C:\Users\YourName\hadoop
set PATH=%HADOOP_HOME%\bin;%HADOOP_HOME%\sbin;%JAVA_HOME%\bin;%PATH%

REM Verify
echo %HADOOP_HOME%
hadoop version
```

Limitation: These variables disappear when you close the terminal.

Method C: Create a Startup Batch Script

Save this as `setup_hadoop.bat` on your Desktop or working folder:

```
@echo off
echo =====
echo    Setting Hadoop Environment Variables
echo =====

set JAVA_HOME=C:\Users\%USERNAME%\tools\jdk1.8.0_xxx
set HADOOP_HOME=C:\Users\%USERNAME%\tools\hadoop
set HADOOP_CONF_DIR=%HADOOP_HOME%\etc\hadoop
set PATH=%HADOOP_HOME%\bin;%HADOOP_HOME%\sbin;%JAVA_HOME%\bin;%PATH%

echo JAVA_HOME    = %JAVA_HOME%
echo HADOOP_HOME  = %HADOOP_HOME%
echo.
echo Environment ready! You can now use Hadoop commands.
echo.

REM Stay in command prompt
cmd /k
```

Usage: Double-click `setup_hadoop.bat` every time you want to work with Hadoop.

2. Install Everything in Your User Directory

Since you can't write to `C:\Program Files` or `C:\`, install everything under your home folder:

```
C:\Users\YourName\tools\
```

```
├─ jdk1.8.0_xxx\  
├─ hadoop\  
│   ├─ bin\  
│   ├─ etc\hadoop\  
│   ├─ sbin\  
│   ├─ data\  
│   │   ├─ namenode\  
│   │   └─ datanode\  
│   └─ tmp\  
└─
```

Modified Configuration Files

core-site.xml

```
<configuration>  
  <property>  
    <name>fs.defaultFS</name>  
    <value>hdfs://localhost:9000</value>  
  </property>  
  <property>  
    <name>hadoop.tmp.dir</name>  
    <value>C:/Users/YourName/tools/hadoop/tmp</value>  
  </property>  
</configuration>
```

hdfs-site.xml

```
<configuration>  
  <property>  
    <name>dfs.replication</name>  
    <value>1</value>  
  </property>  
  <property>  
    <name>dfs.namenode.name.dir</name>  
    <value>file:///C:/Users/YourName/tools/hadoop/data/namenode</value>  
  </property>  
  <property>  
    <name>dfs.datanode.data.dir</name>
```



```
<value>file:///C:/Users/YourName/tools/hadoop/data/datanode</value>
</property>
</configuration>
```

3. Handle `winutils.exe` Without Admin

Normally you'd copy `hadoop.dll` to `System32`, but without admin:

Solution: Keep it in Hadoop's bin folder only

```
C:\Users\YourName\tools\hadoop\bin\winutils.exe
```

```
C:\Users\YourName\tools\hadoop\bin\hadoop.dll
```

Then set this additional variable:

```
set HADOOP_HOME=C:\Users\YourName\tools\hadoop
```

Hadoop will find `winutils.exe` in `%HADOOP_HOME%\bin` automatically – **no need for System32**.

4. Python Setup Without Admin

Install Python Packages Without Admin

```
# Install to user directory (no admin needed)
pip install --user hdfs
pip install --user pandas
```

Python: Set Environment Variables Programmatically

```
import os
import subprocess

# ===== SET ENVIRONMENT VARIABLES IN PYTHON =====
os.environ['JAVA_HOME'] = r'C:\Users\YourName\tools\jdk1.8.0_xxx'
os.environ['HADOOP_HOME'] = r'C:\Users\YourName\tools\hadoop'
os.environ['PATH'] = (
```

```

os.environ['HADOOP_HOME'] + r'\bin;' +
os.environ['HADOOP_HOME'] + r'\sbin;' +
os.environ['JAVA_HOME'] + r'\bin;' +
os.environ['PATH']
)

# ===== VERIFY =====
result = subprocess.run('hadoop version', shell=True,
                        capture_output=True, text=True)
print(result.stdout)

# ===== USE HDFS COMMANDS =====
def hdfs_command(cmd):
    full_cmd = f"hdfs dfs {cmd}"
    result = subprocess.run(full_cmd, shell=True,
                            capture_output=True, text=True)

    if result.stdout:
        print(result.stdout)
    if result.stderr:
        print("Error:", result.stderr)
    return result

# Now use HDFS normally
hdfs_command("-ls /")
hdfs_command("-mkdir /user/test")
hdfs_command(f"-put C:\\\\Users\\YourName\\sample.txt /user/test/")
hdfs_command("-cat /user/test/sample.txt")

```

Python: Full WebHDFS Without Shell (No Environment Variables Needed)

```

"""
This approach uses ONLY HTTP requests - no hadoop binary needed!
You only need the HDFS cluster to be running (e.g., on a remote server).
"""

import requests
import json
import pandas as pd
from io import StringIO

class HDFSClient:
    """Pure Python HDFS client using WebHDFS REST API"""

```

```

def __init__(self, namenode_host="localhost", port=9870, user="hadoop"):
    self.base_url = f"http://{namenode_host}:{port}/webhdfs/v1"
    self.user = user

def _url(self, path, op, **kwargs):
    params = f"?op={op}&user.name={self.user}"
    for key, value in kwargs.items():
        params += f"&{key}={value}"
    return f"{self.base_url}{path}{params}"

def list_dir(self, path="/"):
    """List directory contents"""
    url = self._url(path, "LISTSTATUS")
    response = requests.get(url)
    data = response.json()
    files = data['FileStatuses']['FileStatus']
    for f in files:
        print(f"{'d' if f['type']=='DIRECTORY' else '-'} "
              f"{f['permission']} {f['length']:>10} {f['pathSuffix']}")
    return files

def mkdir(self, path):
    """Create directory"""
    url = self._url(path, "MKDIRS")
    response = requests.put(url)
    result = response.json()
    print(f"Directory created: {result['boolean']}")
    return result['boolean']

def upload(self, hdfs_path, local_path):
    """Upload file to HDFS"""
    url = self._url(hdfs_path, "CREATE", overwrite="true")
    # Step 1: Get redirect
    response = requests.put(url, allow_redirects=False)
    redirect_url = response.headers['Location']
    # Step 2: Upload data
    with open(local_path, 'rb') as f:
        response2 = requests.put(redirect_url, data=f)
    print(f"Upload status: {response2.status_code}")
    return response2.status_code == 201

def read(self, hdfs_path):
    """Read file content"""
    url = self._url(hdfs_path, "OPEN")

```

```

        response = requests.get(url)
        return response.text

def download(self, hdfs_path, local_path):
    """Download file from HDFS"""
    content = self.read(hdfs_path)
    with open(local_path, 'w', encoding='utf-8') as f:
        f.write(content)
    print(f"Downloaded to: {local_path}")

def delete(self, path, recursive=False):
    """Delete file or directory"""
    url = self._url(path, "DELETE", recursive=str(recursive).lower())
    response = requests.delete(url)
    result = response.json()
    print(f"Deleted: {result['boolean']}")
    return result['boolean']

def status(self, path):
    """Get file/directory status"""
    url = self._url(path, "GETFILESTATUS")
    response = requests.get(url)
    return response.json()['FileStatus']

def rename(self, src_path, dest_path):
    """Rename/move file"""
    url = self._url(src_path, "RENAME", destination=dest_path)
    response = requests.put(url)
    result = response.json()
    print(f"Renamed: {result['boolean']}")
    return result['boolean']

def write_text(self, hdfs_path, text):
    """Write text directly to HDFS"""
    url = self._url(hdfs_path, "CREATE", overwrite="true")
    response = requests.put(url, allow_redirects=False)
    redirect_url = response.headers['Location']
    response2 = requests.put(redirect_url,
                             data=text.encode('utf-8'))
    print(f"Write status: {response2.status_code}")
    return response2.status_code == 201

def write_df(self, hdfs_path, df):
    """Write pandas DataFrame as CSV to HDFS"""

```

```

        csv_text = df.to_csv(index=False)
        return self.write_text(hdfs_path, csv_text)

    def read_df(self, hdfs_path):
        """Read CSV from HDFS into pandas DataFrame"""
        content = self.read(hdfs_path)
        return pd.read_csv(StringIO(content))

# ===== USAGE =====
client = HDFSClient(namenode_host="localhost", port=9870, user="hadoop")

# List root
client.list_dir("/")

# Create directory
client.mkdir("/user/nonadmin_data")

# Write text directly
client.write_text("/user/nonadmin_data/hello.txt",
                  "Hello from Python without admin!")

# Read file
content = client.read("/user/nonadmin_data/hello.txt")
print("Content:", content)

# Upload local file
client.upload("/user/nonadmin_data/local_file.txt",
              r"C:\Users\YourName\Desktop\myfile.txt")

# Work with DataFrames
df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Score': [95, 87, 92]
})
client.write_df("/user/nonadmin_data/scores.csv", df)

df_back = client.read_df("/user/nonadmin_data/scores.csv")
print(df_back)

# Clean up
client.delete("/user/nonadmin_data", recursive=True)

```

5. R Setup Without Admin

Install R Packages to User Library

```
# Create a personal library if prompted
# R will ask: "Would you like to use a personal library instead?" → Yes

install.packages("httr", lib = Sys.getenv("R_LIBS_USER"))
install.packages("jsonlite", lib = Sys.getenv("R_LIBS_USER"))
```

R: Set Environment Variables Programmatically

```
# ===== SET ENVIRONMENT INSIDE R SESSION =====
Sys.setenv(JAVA_HOME = "C:/Users/YourName/tools/jdk1.8.0_xxx")
Sys.setenv(HADOOP_HOME = "C:/Users/YourName/tools/hadoop")

old_path <- Sys.getenv("PATH")
new_path <- paste0(
  Sys.getenv("HADOOP_HOME"), "/bin;",
  Sys.getenv("HADOOP_HOME"), "/sbin;",
  Sys.getenv("JAVA_HOME"), "/bin;",
  old_path
)
Sys.setenv(PATH = new_path)

# Verify
system("hadoop version", intern = TRUE)
```

R: Pure WebHDFS (No Hadoop Install Needed on Client)

```
library(httr)
library(jsonlite)

# ===== HDFS CLIENT CLASS (FUNCTIONAL STYLE) =====
hdfs_config <- list(
  base_url = "http://localhost:9870/webhdfs/v1",
  user = "hadoop"
)
```

```

hdfs_url <- function(path, op, ...) {
  params <- list(op = op, user.name = hdfs_config$user, ...)
  param_str <- paste(names(params), params, sep = "=", collapse = "&")
  paste0(hdfs_config$base_url, path, "?", param_str)
}

# ===== LIST DIRECTORY =====
hdfs_ls <- function(path = "/") {
  url <- hdfs_url(path, "LISTSTATUS")
  resp <- GET(url)
  data <- fromJSON(content(resp, "text", encoding = "UTF-8"))
  files <- data$FileStatuses$FileStatus
  if (!is.null(files)) {
    cat(sprintf("%-5s %-10s %10s %s\n", "Type", "Permission", "Size", "Name"),
        cat(strrep("-", 50), "\n"))
    for (i in seq_len(nrow(files))) {
      cat(sprintf("%-5s %-10s %10s %s\n",
                  files$type[i], files$permission[i],
                  files$length[i], files$pathSuffix[i]))
    }
  }
  return(files)
}

# ===== CREATE DIRECTORY =====
hdfs_mkdir <- function(path) {
  url <- hdfs_url(path, "MKDIRS")
  resp <- PUT(url)
  result <- fromJSON(content(resp, "text", encoding = "UTF-8"))
  cat("Directory created:", result$boolean, "\n")
  return(result$boolean)
}

# ===== WRITE TEXT =====
hdfs_write <- function(path, text_content) {
  url <- hdfs_url(path, "CREATE", overwrite = "true")
  resp <- PUT(url, config = config(followlocation = FALSE))
  redirect <- headers(resp)$location
  resp2 <- PUT(redirect, body = charToRaw(text_content),
              content_type("application/octet-stream"))
  cat("Write status:", status_code(resp2), "\n")
  return(status_code(resp2) == 201)
}

```

```

# ===== READ FILE =====
hdfs_read <- function(path) {
  url <- hdfs_url(path, "OPEN")
  resp <- GET(url)
  return(content(resp, "text", encoding = "UTF-8"))
}

# ===== UPLOAD LOCAL FILE =====
hdfs_upload <- function(local_path, hdfs_path) {
  url <- hdfs_url(hdfs_path, "CREATE", overwrite = "true")
  resp <- PUT(url, config = config(followlocation = FALSE))
  redirect <- headers(resp)$location
  resp2 <- PUT(redirect, body = upload_file(local_path))
  cat("Upload status:", status_code(resp2), "\n")
  return(status_code(resp2) == 201)
}

# ===== DOWNLOAD FILE =====
hdfs_download <- function(hdfs_path, local_path) {
  content_text <- hdfs_read(hdfs_path)
  writeLines(content_text, local_path)
  cat("Downloaded to:", local_path, "\n")
}

# ===== DELETE =====
hdfs_delete <- function(path, recursive = FALSE) {
  url <- hdfs_url(path, "DELETE", recursive = tolower(as.character(recursive)))
  resp <- DELETE(url)
  result <- fromJSON(content(resp, "text", encoding = "UTF-8"))
  cat("Deleted:", result$boolean, "\n")
  return(result$boolean)
}

# ===== FILE STATUS =====
hdfs_status <- function(path) {
  url <- hdfs_url(path, "GETFILESTATUS")
  resp <- GET(url)
  status <- fromJSON(content(resp, "text", encoding = "UTF-8"))
  return(status$FileStatus)
}

# ===== WRITE DATAFRAME =====
hdfs_write_csv <- function(df, hdfs_path) {
  tmp <- tempfile(fileext = ".csv")

```



```

write.csv(df, tmp, row.names = FALSE)
csv_text <- paste(readLines(tmp), collapse = "\n")
unlink(tmp)
hdfs_write(hdfs_path, csv_text)
}

# ===== READ DATAFRAME =====
hdfs_read_csv <- function(hdfs_path) {
  content_text <- hdfs_read(hdfs_path)
  read.csv(text = content_text, stringsAsFactors = FALSE)
}

# ===== DEMONSTRATION =====
# List root
hdfs_ls("/")

# Create directory
hdfs_mkdir("/user/r_nonadmin")

# Write directly
hdfs_write("/user/r_nonadmin/hello.txt", "Hello from R without admin rights")

# Read back
cat("Content:", hdfs_read("/user/r_nonadmin/hello.txt"), "\n")

# DataFrame operations
employees <- data.frame(
  Name = c("Alice", "Bob", "Charlie"),
  Age = c(25, 30, 35),
  Dept = c("HR", "IT", "Finance")
)

hdfs_write_csv(employees, "/user/r_nonadmin/employees.csv")
df_back <- hdfs_read_csv("/user/r_nonadmin/employees.csv")
print(df_back)

# Clean up
hdfs_delete("/user/r_nonadmin", recursive = TRUE)

```

6. Summary: Approaches Without Admin

Challenge	Solution
Can't set system PATH	Use user-level variables or <code>set</code> / <code>setx</code> commands
Can't install to Program Files	Install under <code>C:\Users\YourName\tools\</code>
Can't copy to System32	Keep <code>hadoop.dll</code> in <code>%HADOOP_HOME%\bin</code> only
Can't install pip packages globally	Use <code>pip install --user</code>
No admin for R packages	Install to personal R library
Can't install Hadoop at all	Use WebHDFS REST API (Python <code>requests</code> / R <code>httr</code>) – only needs network access to the cluster

Best Recommendation for Non-Admin Users

The **WebHDFS REST API** approach (shown in the Python `HDFSClient` class and R functions above) is the most practical because it:

- **Requires zero local installation** of Hadoop
- **Needs no environment variables**
- Only requires `requests` (Python) or `httr` (R) – lightweight packages
- Works from **any machine** that has network access to the HDFS NameNode

The only prerequisite is that the HDFS cluster is running and WebHDFS is enabled (port **9870** is accessible).

Would you like me to also show how to set this up using **Docker** (which can run without admin on some configurations) as an alternative way to get a fully functional Hadoop cluster running locally?
